

**ПРАВИТЕЛЬСТВО РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
«ВЫСШАЯ ШКОЛА ЭКОНОМИКИ»**

Факультет компьютерных наук

Образовательная программа «Дизайн и разработка информационных продуктов»

СОГЛАСОВАНО

Научный руководитель
Приглашённый преподаватель:
Факультет компьютерных наук /
Департамент больших данных и
информационного поиска / Базовая
кафедра Т-Банка

_____ А. А. Шкель
«__» _____ 2026 г.

УТВЕРЖДЕНО

Академический руководитель
образовательной программы
«Дизайн и разработка
информационных продуктов»,
преподаватель базовой кафедры Т-
Банка

_____ Д. Д. Син
«__» _____ 2026 г.

ПЛАТФОРМА ДЛЯ NO-CODE АВТОМАТИЗАЦИИ БИЗНЕС-ПРОЦЕССОВ.

Техническое задание

ЛИСТ УТВЕРЖДЕНИЯ

RU.17701729.09.03-04 ТЗ 01-1-ЛУ

Исполнители:

Студент группы БДРИП241

_____ / С. В. Гаврилин /
«__» _____ 2026 г.

Студент группы БДРИП242

_____ / А. А. Клушин /
«__» _____ 2026 г.

Москва 2026

Инд.№ подп	Подп. и дата	Взам. инв.№	Инд.№ дубл.	Подп. и дата

УТВЕРЖДЕН

RU.17701729.09.03-04 ТЗ 01-1-ЛУ

ПЛАТФОРМА ДЛЯ NO-CODE АВТОМАТИЗАЦИИ БИЗНЕС-ПРОЦЕССОВ.

Техническое задание

RU.17701729.09.03-04 ТЗ 01-1

Листов 19

Москва 2026

Инов.№ подп	Подп. и дата	Взам. инв.№	Инов.№ дубл.	Подп. и дата

АННОТАЦИЯ

Техническое задание – это основной документ, оговаривающий набор требований и порядок создания программного продукта, в соответствии с которым производится разработка программы ее тестирование и приемка.

Настоящее Техническое задание на разработку «Платформа для no-code автоматизации бизнес-процессов» содержит следующие разделы: «Введение», «Основания для разработки», «Назначение разработки», «Требования к программе», «Требования к программным документам», «Технико-экономические показатели», «Стадии и этапы разработки», «Порядок контроля и приемки», приложения [5].

В разделе «Введение» указано наименование и краткая характеристика области применения программы.

В разделе «Основания для разработки» указан документ, на основании которого ведется разработка, и наименование темы разработки.

В разделе «Назначение разработки» указано функциональное и эксплуатационное назначение создаваемого программного продукта.

Раздел «Требования к программе» содержит указание на основные требования к функциональным характеристикам программы, к её надежности и к условиям эксплуатации, к составу и параметрам технических средств, к информационной и программной совместимости, к транспортировке и хранению.

Раздел «Требования к программным документам» содержит указание на предварительный состав программной документации и специальные требования к ней.

Раздел «Технико-экономические показатели» содержит информацию об ориентировочной экономической эффективности разработки, экономические преимущества разработки программы.

Раздел «Стадии и этапы разработки» содержит информацию о стадиях разработки, этапах и содержании работ.

В разделе «Порядок контроля и приемки» указаны общие требования к приемке работы.

Настоящий документ разработан в соответствии с требованиями:

1. ГОСТ 19.103-77 [1]: Обозначения программ и программных документов.
2. ГОСТ 19.104-78 [2]: Основные надписи.
3. ГОСТ 19.105-78 [3]: Общие требования к программным документам.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.03-04 ТЗ 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

RU.17701729.09.03-04 ТЗ 01-1

4. ГОСТ 19.106-78 [4]: Требования к программным документам, выполненным печатным способом.
5. ГОСТ 19.201-78 [5]: Техническое задание. Требования к содержанию и оформлению.

Изменения к данному Техническому заданию оформляются согласно ГОСТ 19.603-78, ГОСТ 19.604-78.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.03-04 ТЗ 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

СОДЕРЖАНИЕ

1. ВВЕДЕНИЕ	6
1.1. Наименование программы	6
1.2. Краткая характеристика области применения	6
2. ОСНОВАНИЯ ДЛЯ РАЗРАБОТКИ	7
3. НАЗНАЧЕНИЕ РАЗРАБОТКИ	8
3.1. Функциональное назначение	8
3.2. Эксплуатационное назначение	8
4. ТРЕБОВАНИЯ К ПРОГРАММЕ	9
4.1. Требования к функциональным характеристикам	9
4.1.1. Требования к составу выполняемых функций	9
4.1.2. Организация входных данных	11
4.1.3. Организация выходных данных	11
4.2. Требования к временным характеристикам	12
4.3. Требования к надёжности	12
4.4. Условия эксплуатации	12
4.4.1. Климатические условия	12
4.4.2. Технические средства	13
4.5. Требования к информационной и программной совместимости	13
4.5.1. Языки и каркасы	13
4.5.2. Программные средства	13
4.6. Требования к транспортировке и хранению	13
5. ТРЕБОВАНИЯ К ПРОГРАММНОЙ ДОКУМЕНТАЦИИ	14
5.1. Состав программной документации	14
5.2. Специальные требования	14
6. ТЕХНИКО-ЭКОНОМИЧЕСКИЕ ПОКАЗАТЕЛИ	15
6.1. Ориентировочная экономическая эффективность	15
6.2. Предполагаемая потребность	15
6.3. Экономические преимущества по сравнению с аналогами	15
7. СТАДИИ И ЭТАПЫ РАЗРАБОТКИ	16
8. ПОРЯДОК КОНТРОЛЯ И ПРИЁМКИ	17

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.03-04 ТЗ 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

8.1. Виды испытаний	17
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	18

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.03-04 ТЗ 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

1. ВВЕДЕНИЕ

1.1. Наименование программы

Наименование темы разработки — «Платформа для no-code автоматизации бизнес-процессов».

Наименование на английском языке — «No-code business process automation platform».

Внутреннее имя продукта команды — FluxPilot.

1.2. Краткая характеристика области применения

Разрабатываемая программа представляет собой веб-платформу, предоставляющую возможность визуального проектирования и автоматического исполнения бизнес-процессов (workflow) без необходимости написания программного кода со стороны конечного пользователя. Платформа ориентирована на решение задач интеграции между внешними и внутренними информационными системами через HTTP-запросы; обработки, фильтрации и трансформации потоков данных; реагирования на события по расписанию (cron) или через входящие HTTP-запросы (webhook); проведения A/B-экспериментов с разделением пользовательского трафика. Движок исполнения workflow строится по концепции, совместимой с существующими решениями (в частности, p8n — концепция узлов и связей), однако дополнен механизмами, отсутствующими во многих аналогах: двухуровневым версионированием с откатом, snapshot-ами с именованными тегами, контент-адресуемым хранением конфигов для дедупликации, изолированным исполнением пользовательского кода в одноразовых Docker-контейнерах, отладочным интерфейсом с трансляцией событий через WebSocket, feature-флагами для управления функциональными блоками без повторного развёртывания.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.03-04 ТЗ 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

2. ОСНОВАНИЯ ДЛЯ РАЗРАБОТКИ

Разработка ведётся на основании учебного плана подготовки бакалавров по образовательной программе «Дизайн и разработка информационных продуктов» факультета компьютерных наук Национального исследовательского университета «Высшая школа экономики» и утверждённой академическим руководителем программы темы курсового проекта (контрольная точка 1 курсового проекта зафиксирована и одобрена научным руководителем).

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.03-04 ТЗ 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

3. НАЗНАЧЕНИЕ РАЗРАБОТКИ

3.1. Функциональное назначение

Разрабатываемый программный продукт предназначен для визуального проектирования, конфигурирования и автоматического исполнения бизнес-процессов. Продукт состоит из двух функциональных частей: серверной (Kotlin 2.2 на Spring Boot 3.5.3, JDK 24) и клиентской (Angular 19.2 на TypeScript 5.6). Программный продукт обеспечивает выполнение следующих основных функций:

- визуальное создание и редактирование workflow-графов через веб-интерфейс;
- исполнение workflow с поддержкой параллельного выполнения независимых ветвей и обработки сбоев;
- запуск workflow посредством трёх типов триггеров: ручной запуск, scheduler (cron), webhook;
- обработка данных через узлы потоков (filter, map, reduce, foreach, flatmap);
- исполнение произвольного Python-кода в изолированной Docker-песочнице (Code Node);
- мониторинг хода исполнения в реальном времени через WebSocket-канал (STOMP поверх SockJS);
- двухуровневое версионирование workflow с возможностью отката к ранее сохранённым состояниям;
- контент-адресуемое хранение конфигов в MinIO с дедупликацией по SHA-256;
- автоматическая документация REST API через springdoc-openapi (OpenAPI 3.0 + Swagger UI);
- встроенный модуль продуктовой аналитики с подсчётом доверительных интервалов конверсии для A/B-экспериментов.

3.2. Эксплуатационное назначение

Программный продукт предназначен для использования: бизнес-аналитиками и продуктовыми менеджерами, нуждающимися в автоматизации повторяющихся задач интеграции и проведении A/B-экспериментов без привлечения команды разработки; системными администраторами и DevOps-инженерами, разворачивающими платформу в инфраструктуре организации; разработчиками, использующими платформу как gateway-инструмент для сложных конвейеров обработки данных.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.03-04 ТЗ 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

4. ТРЕБОВАНИЯ К ПРОГРАММЕ

4.1. Требования к функциональным характеристикам

4.1.1. Требования к составу выполняемых функций

Для проекта «Платформа для no-code автоматизации бизнес-процессов» программный продукт должен реализовывать следующие функции:

1) **Визуальный редактор workflow.** Должен быть реализован интерактивный холст (canvas) с поддержкой: drag-and-drop размещения узлов из палитры; рисования направленных рёбер между выходными и входными портами узлов в виде SVG-кривых Безье с указателем направления; перемещения существующих узлов с автоматическим обновлением подключённых рёбер; зума колесом мыши с центрированием в позиции курсора; панорамирования (pan) drag-and-drop пустого пространства; селекции и групповой селекции узлов; удаления выделенных узлов через клавишу Delete; сохранения и загрузки конфигурации workflow из серверной части через REST API; debounced-save с задержкой 1 секунда после последнего изменения.

2) **Система триггеров.** Должны быть реализованы три типа триггеров: ручной запуск через REST-эндпойнт; scheduler с cron-выражениями (формат Spring, 6 полей: секунды, минуты, часы, день месяца, месяц, день недели); webhook со сгенерированным уникальным URL вида /v1/webhook/{token}.

3) **HTTP-узел.** Должен быть реализован исполнитель узла, выполняющий HTTP-запросы с поддержкой методов GET, POST, PUT, PATCH, DELETE; настраиваемых заголовков (как пар ключ-значение); тела запроса в формате JSON; настраиваемого тайм-аута (по умолчанию 30 секунд, максимум 300); корректной обработки ошибочных ситуаций (DNS, connection, тайм-аут, HTTP-статусы 4xx/5xx).

4) **Узлы потоков данных.** Должны быть реализованы пять операций обработки списков: filter (фильтрация по булевому предикату), map (применение трансформации к каждому элементу), reduce (агрегация с начальным значением), foreach (выполнение side-эффектного действия для каждого элемента с возвратом исходного списка), flatmap (разворачивание вложенных списков).

5) **Code Node с песочницей.** Должен быть реализован исполнитель пользовательского Python- и JavaScript-кода в одноразовом Docker-контейнере с полной изоляцией: --network none (отсутствие сетевого доступа), --read-only (read-only корневая FS), --tmpfs /tmp:rw,size=16m (16 МБ writable временной памяти), --cap-drop ALL (drop всех Linux capabilities), --security-opt no-new-privileges (запрет escalation), --memory 256m (лимит ОЗУ), --cpus 1 (лимит CPU),

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.03-04 ТЗ 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

--pids-limit 64 (лимит процессов), тайм-аут исполнения 5 секунд (плюс 30 секунд overhead на запуск контейнера).

6) **Параллельное исполнение.** Должна быть реализована возможность одновременного выполнения независимых ветвей workflow с пулом не менее 16 потоков; при сбое одного узла зависимые узлы получают статус skipped с диагностическим сообщением, параллельные ветви продолжают исполнение; перед запуском выполняется проверка графа на отсутствие циклов через топологическую сортировку по алгоритму Кана.

7) **Двухуровневое версионирование.** Должна быть реализована двухуровневая схема: автоматические ревизии (создаются при каждом сохранении конфигурации) и именованные версии-снапшоты (создаются по явному запросу пользователя с указанием тега). Должна быть обеспечена возможность отката к произвольной ревизии или версии; иммутабельность исторических ревизий (ON DELETE RESTRICT на FK ссылающихся версий); append-only-семантика операции отката (создание новой ревизии с прошлым graph_skeleton).

8) **Интерфейс просмотра состояния.** Должен быть реализован интерфейс отображения хода исполнения: входные и выходные данные каждого узла, диагностические сообщения об ошибках, история всех запусков с временами и длительностями.

9) **Feature-флаги.** Должна быть обеспечена возможность управляемой раскатки функциональности на подмножество пользовательского трафика без повторного развёртывания. Функциональность реализуется на базе узла разделения потока (BranchSplit, см. требование 12) в режиме «Feature Flag» со стратегиями attribute (отбор по атрибутам входного объекта — например, идентификатору пользователя или географическому полю) и percentage (процентная раскатка по детерминированному хешу с фиксированной солью); включение/отключение и изменение долей раскатки выполняется через инспектор узла без перезапуска платформы.

10) **WebSocket-трансляция событий.** Должна быть реализована трансляция событий жизненного цикла исполнения (workflow_started, node_reached, node_action, node_exited, workflow_finished) клиентским подписчикам через WebSocket-канал с протоколом STOMP поверх SockJS-транспорта; топик /topic/workflows/{workflowId}/graph.

11) **Контент-адресуемое хранение в MinIO.** Должна быть реализована схема хранения конфигов узлов в объектном хранилище MinIO по SHA-256-хешу содержимого; одинаковые конфиги между разными ревизиями и workflow не дублируются; вспомогательная таблица blob_index в реляционной БД поддерживает индекс существующих хешей.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.03-04 ТЗ 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

12) **A/B-аналитика workflow.** Должны быть реализованы специализированные узлы разделения потока на N вариантов по выбираемой стратегии (random — случайное по весам, hash — детерминированное по хешу идентификатора, modulo — по остатку, attribute — по правилам атрибутов входного объекта, percentage — процентная раскатка, stratified — стратифицированное разделение) с режимом split (разделение потока) или pick (выбор одной ветви, см. требование 9) и парного слияния ветвей; серверный модуль аналитики с агрегацией просмотров и конверсий по вариантам, статистической проверкой значимости разности конверсий (z-тест двух пропорций) и расчётом доверительных интервалов; REST-эндпойнт получения сводных метрик по workflow для отображения в клиентской панели аналитики.

13) **Снапшоты workflow.** Должен быть реализован механизм именованных снапшотов уровня документа: отдельная таблица с привязкой к конкретной ревизии, REST-эндпойнты создания / чтения / удаления, возможность задать пользовательские имя и описание снапшота.

14) **Режим пошаговой отладки workflow.** Должна быть обеспечена возможность запуска одной выбранной ноды workflow с произвольным входным значением, без полного прогона графа. Серверная часть должна предоставлять соответствующий механизм исполнения, использующий ту же реестровую инфраструктуру исполнителей и тот же механизм трансляции событий, что и штатное исполнение workflow. Результат отладочного запуска должен возвращаться клиентской части в синхронном виде и фиксироваться в истории запусков с пометкой отладочного исполнения. Клиентская часть должна предоставлять интерфейс выбора целевой ноды на холсте редактора, ввода входного значения и инициирования отладочного запуска.

4.1.2. Организация входных данных

Входные данные серверной части передаются: по протоколу HTTP в виде JSON-тел через REST API (запросы от клиентской части и webhook-источников); через визуальный редактор клиентской части (где пользователь конфигурирует параметры узлов через формы инспектора); через HTTP-ответы внешних сервисов, получаемые HTTP-узлами при исполнении workflow. Метаданные хранятся в PostgreSQL в типизированных колонках. Структура workflow (граф узлов и связей) хранится в jsonb-колонках. Тяжёлые конфиги узлов — в MinIO по content-addressed-схеме.

4.1.3. Организация выходных данных

Выходные данные формируются: в виде JSON-ответов REST API; в виде событий WebSocket для трансляции хода исполнения клиентам; в виде сохранённых результатов каждого

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.03-04 ТЗ 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

узла в БД (поля `input_json`, `output_json`, `error_message` таблицы `node_run`) и агрегированного результата `workflow_run`.

4.2. Требования к временным характеристикам

- время отклика веб-интерфейса при типовых операциях — не более 2 секунд;
- время от приёма триггера до перехода первого узла `workflow` в статус `running` — не более 5 секунд (с учётом `cold-start` песочницы Code Node при первом запуске);
- 95-й перцентиль REST API на чтение — не более 500 мс при типовой нагрузке (10 одновременных подключений, пустая база и база со 100 `workflow` / 1 000 запусков);
- 95-й перцентиль WebSocket-задержки в локальной сети — не более 200 мс;
- `debounced-save` в редакторе — задержка сохранения не более 1 секунды после последнего изменения;
- время полного прогона CI-пайплайна на `pull request` — не более 12 минут;
- готовность системы после `docker compose up -d` — не более 60 секунд (после первого `pull` Docker-образов).

4.3. Требования к надёжности

Программный продукт должен быть устойчив к некорректным входным данным: валидация на уровне DTO через Spring Validation; валидация на уровне форм клиентской части через Angular Reactive Forms; единообразная обработка ошибок через `RestExceptionHandler` с возвратом машинно-читаемых `errorCode` и человекочитаемых `message`. Code Node должен исполняться исключительно в Docker-песочнице с полной изоляцией. При отказе одного узла `workflow` данные запуска не должны утрачиваться: статус, входы и выходы всех узлов фиксируются в БД. Принудительное завершение `backend` (`docker stop`) не должно приводить к повреждению целостности БД.

4.4. Условия эксплуатации

4.4.1. Климатические условия

Климатические условия эксплуатации программного продукта должны соответствовать требованиям к техническим средствам, на которых разворачивается серверная часть и используется клиентская часть.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.03-04 ТЗ 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

4.4.2. Технические средства

Сервер: процессор не менее 4 ядер архитектуры x86_64; оперативная память не менее 8 ГБ (рекомендуется 16 ГБ); дисковое пространство не менее 20 ГБ; стабильное подключение к сети Интернет. Программные средства: JDK 24, PostgreSQL 16, MinIO, Docker Engine 20.10+.

Клиент: современный браузер (Chrome 90+, Firefox 90+, Safari 15+, Edge 90+); разрешение экрана не менее 1280×720 (рекомендуется 1920×1080); включённая поддержка JavaScript и WebSocket.

4.5. Требования к информационной и программной совместимости

4.5.1. Языки и каркасы

Сервер: Kotlin 2.2.21, Spring Boot 3.5.3, JDK 24, Apache Maven 3.9+.

Клиент: TypeScript 5.6, Angular 19.2, RxJS 7.8, Node.js 20 LTS.

4.5.2. Программные средства

PostgreSQL 16; Liquibase 4 (миграции схемы); MinIO (объектное хранилище, S3-совместимое); Docker Engine 20.10+ (контейнеризация + Code Node-песочница); springdoc-openapi 2.7.0 (документация API); JUnit 5, Testcontainers 1.21.1, JaCoCo 0.8.13 (тестирование backend); Jasmine, Karma, Playwright 1.59.1 (тестирование frontend).

4.6. Требования к транспортировке и хранению

Программный продукт распространяется в виде исходного кода в Git-репозитории (github.com/arklual/cursach) и Docker-образов в GitHub Container Registry (ghcr.io). Развёртывание осуществляется через `docker compose -f deploy/docker-compose.prod.yml up -d` либо ручной запуск fat-JAR backend и собранного бандла frontend.

Развёрнутая (промышленная) инстанция программы публично доступна по адресу <https://fluxpilot.ru/> и используется для демонстрации и приёмочных испытаний.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.03-04 ТЗ 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

5. ТРЕБОВАНИЯ К ПРОГРАММНОЙ ДОКУМЕНТАЦИИ

5.1. Состав программной документации

В состав программной документации входят следующие документы:

- **Техническое задание** (общая часть, шифр RU.17701729.09.03-04 ТЗ 01-1) и две индивидуальные части (ТЗ 02-1 — клиентская часть, Гаврилин С.В.; ТЗ 03-1 — серверная часть, Клушин А.А.) в соответствии с ГОСТ 19.201-78;
- **Пояснительная записка** (общая часть, шифр RU.17701729.09.03-04 81 01-1) и две индивидуальные части (81 02-1, 81 03-1) в соответствии с ГОСТ 19.404-79;
- **Программа и методика испытаний** (общая часть, шифр RU.17701729.09.03-04 51 01-1) и две индивидуальные части (51 02-1, 51 03-1) в соответствии с ГОСТ 19.301-79;
- **Руководство оператора** (общая часть, шифр RU.17701729.09.03-04 34 01-1) и две индивидуальные части (34 02-1, 34 03-1) в соответствии с ГОСТ 19.505-79;
- **Текст программы** (общая часть, шифр RU.17701729.09.03-04 12 01-1) и две индивидуальные части (12 02-1, 12 03-1) в соответствии с ГОСТ 19.401-78.

5.2. Специальные требования

Документы оформляются в соответствии с ГОСТ 19.106-78. Документация сдаётся в формате PDF.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.03-04 ТЗ 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

6. ТЕХНИКО-ЭКОНОМИЧЕСКИЕ ПОКАЗАТЕЛИ

6.1. Ориентировочная экономическая эффективность

Данный курсовой проект имеет учебную природу и не предусматривает прямого расчёта экономической эффективности по нормативным методикам. Косвенная оценка прикладной эффективности при применении в реальной организации приведена в общей пояснительной записке (раздел «Технико-экономические показатели»).

6.2. Предполагаемая потребность

Платформа будет востребована компаниями малого и среднего бизнеса, крупными организациями с продуктовыми и аналитическими командами, организациями с требованиями self-hosted развёртывания (банки, государственные структуры, оборонная промышленность).

6.3. Экономические преимущества по сравнению с аналогами

Рассмотрено девять продуктов-аналогов: n8n, Zapier, Make, Apache Airflow, Node-RED, Prefect, Temporal, Microsoft Power Automate, Camunda. Детальный анализ — в общей пояснительной записке (раздел 3.1.1). Разрабатываемый продукт занимает нишу между лёгкими SaaS-конструкторами (Zapier, Make, Power Automate) и тяжёлыми платформами оркестрации (Airflow, Prefect, Temporal): обеспечивает визуальный редактор для непрограммистов и одновременно реализует production-ready движок на JVM с настоящим параллелизмом, безопасной Docker-песочницей, двухуровневым версионированием с откатом, content-addressed-дедупликацией.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.03-04 ТЗ 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

7. СТАДИИ И ЭТАПЫ РАЗРАБОТКИ

Стадия	Этап	Содержание работ	Сроки
1. ТЗ	Проектирование	Анализ аналогов, выбор стека, ER-диаграмма, разделение зон ответственности	Декабрь 2025
2. Рабочий проект	Редактор	Холст с drag-and-drop, рисование рёбер SVG, инспектор узлов, сохранение и загрузка	Январь 2026
2. Рабочий проект	Движок	Ядро исполнения (топологическая сортировка + CompletableFuture), HTTP-узел, базовые триггеры	Февраль 2026
2. Рабочий проект	Мониторинг	WebSocket-канал, визуальная индикация в реальном времени, история запусков	Март 2026
2. Рабочий проект	Расширение	Потоки данных, Code Node + Docker-песочница, параллельное исполнение, версионирование, MinIO content-addressed	Апрель 2026
3. Внедрение	Тестирование	Покрытие модульными и интеграционными тестами, end-to-end Playwright, CI-пайплайн, документация	Апрель–май 2026
3. Внедрение	Защита	Финализация документации, презентация и защита курсовой работы	Май 2026

Таблица 1. Стадии и этапы разработки

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.03-04 ТЗ 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

8. ПОРЯДОК КОНТРОЛЯ И ПРИЁМКИ

8.1. Виды испытаний

Контроль и приёмка осуществляются в соответствии с документом «Программа и методика испытаний» (общая часть, шифр RU.17701729.09.03-04 51 01-1), что подразумевает проверку выполнения всех функциональных требований раздела 4.1.1 настоящего ТЗ, достижение целевых нефункциональных показателей, успешное прохождение полного набора автоматических тестов (около 230 тестов: backend модульные около 55, backend интеграционные около 12, frontend модульные около 65, end-to-end около 95; покрытие backend по JaCoCo не менее 90 %).

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.03-04 ТЗ 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Spring Framework Reference Documentation [Электронный ресурс]. — URL: <https://docs.spring.io/spring-framework/reference/> (дата обращения: 03.12.2025).
2. Angular Documentation [Электронный ресурс] / Google. — URL: <https://angular.dev/> (дата обращения: 10.12.2025).
3. PostgreSQL 16 Documentation [Электронный ресурс]. — URL: <https://www.postgresql.org/docs/16/> (дата обращения: 12.12.2025).
4. Docker Engine Documentation [Электронный ресурс]. — URL: <https://docs.docker.com/engine/> (дата обращения: 18.03.2026).
5. MinIO Object Storage Documentation [Электронный ресурс]. — URL: <https://min.io/docs/minio/linux/index.html> (дата обращения: 20.02.2026).
6. n8n Documentation [Электронный ресурс]. — URL: <https://docs.n8n.io/> (дата обращения: 12.11.2025).
7. Apache Airflow Documentation [Электронный ресурс]. — URL: <https://airflow.apache.org/docs/> (дата обращения: 15.11.2025).
8. Camunda Platform Documentation [Электронный ресурс]. — URL: <https://docs.camunda.org/> (дата обращения: 18.11.2025).
9. Клеппман М. Высоконагруженные приложения // Программирование, масштабирование, поддержка. Питер. – 2018.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.03-04 ТЗ 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

ЛИСТ РЕГИСТРАЦИИ ИЗМЕНЕНИЙ

[illegible]